

Тема урока: пользовательские функции

Функции

В предыдущих уроках мы использовали встроенные в Python функции `print()`, `input()`, `int()`, `str()`, `len()` и многие другие. Пришло время начать писать свои собственные функции.

В самом начале курса вам было предложено решить задачу, в которой требовалось изобразить звездный прямоугольник размерами 5×7 (5 строк и 7 столбцов).

Наш первый вариант кода выглядел примерно так:

```
print('*****')
print('*****')
print('*****')
print('*****')
print('*****')
```

Далее мы изучили оператор умножения строки на число (оператор повторения) и написали бы код:

```
print('*' * 7)
print('*' * 7)
print('*' * 7)
print('*' * 7)
print('*' * 7)
```

Ну и наконец, мы изучили циклы, после чего код принял бы вид:

```
for _ in range(5):
    print('*' * 7)
```

А теперь представим, что таких прямоугольников нужно изобразить не один, а несколько, скажем, 3 штуки.

Тогда код программы будет иметь вид:

```
for _ in range(5):
    print('*' * 7)

print()

for _ in range(5):
    print('*' * 7)

print()

for _ in range(5):
    print('*' * 7)
```

Результатом выполнения такого кода будет:

```
*****
*****
*****
*****
*****

*****
*****
*****
*****
*****

*****
*****
*****
*****
*****
```

И хотя предыдущий код полностью решает поставленную задачу, он не лишен недостатков. Во-первых, он довольно громоздкий из-за **повторения части кода**, отвечающей за вывод прямоугольника. Во-вторых, если понадобится изменить размеры прямоугольника, придется менять их трижды, в каждой части кода, выводящей прямоугольник.

Вместо повторения кода для вывода прямоугольника, можно перенести его в отдельную **функцию** и вызвать ее 3 раза.

Для создания функции пишем такой код:

```
def draw_box():
    for _ in range(5):
        print('*' * 7)
```

Когда функция создана, чтобы увидеть результат ее работы, надо вызвать ее по имени:

```
draw_box()
```

Именованние функций

Имена функциям назначаются точно так же, как переменным. Имя функции должно быть достаточно описательным, чтобы любой читающий ваш код, мог догадаться, что именно делает функция.

Python и тут требует соблюдения тех же правил, что при именовании переменных:

1. в имени функции используются только латинские буквы a-z, A-Z, цифры и символ нижнего подчеркивания (_);
2. имя функции не может начинаться с цифры;
3. имя функции по возможности должно отражать ее назначение;
4. символы верхнего и нижнего регистра различаются.



Помни: Python — регистрочувствительный язык. Для именования переменных и функций принято использовать

стиль **lower_case_with_underscores** (слова из маленьких букв с подчеркиваниями).

Поскольку функции **выполняют действия**, большинство программистов предпочитает в именах функций **использовать глаголы**. Например:

- функцию, которая **рисует прямоугольник**, можно назвать `draw_box()` ;
- функцию, которая **печатает чек**, можно назвать `print_check()` ;
- функцию, которая вычисляет заработную плату до удержаний, можно назвать `calculate_gross_pay()` .

Каждое из этих имен дает описание того, что функция делает.

Объявление функции

Итак, **функция – отдельная, функционально независимая часть программы, выполняющая определенную задачу.**

Функции объявляются с помощью ключевого слова `def` (от англ. define – определять). За ключевым словом `def` следуют название функции, круглые скобки `()` и двоеточие `:`.

```
def название_функции():  
    блок кода
```

```
def имя функции () :  
    блок кода
```



Первая строка объявления функции называется **заголовком функции**.

Со следующей строки идет блок кода – **тело функции**. Это набор инструкций, составляющих одно целое и выполняющихся каждый раз, когда вызывается функция.

Обратите внимание, что каждая строка в теле функции выделена отступом.



Для выделения строк блока кода отступом программисты Python обычно используют **четыре пробела** в соответствии со стандартом PEP 8.

Рассмотрим объявление функции:

```
def print_message():  
    print('Я - Артур,')  
    print('король британцев. ')
```

Этот фрагмент кода определяет функцию с именем `print_message()`. Тело ее состоит из двух инструкций, и вызов приведет к их исполнению.

Вызов функции

Для вызова функции пишут ее название и круглые скобки.



Важно: очень часто начинающие программисты забывают вызывать функцию. Помните, что объявление функции не вызывает ее.

```
# объявление функции  
def print_message():  
    print('Я - Артур,')  
    print('король британцев. ')  
  
# вызов функции  
print_message()
```

Примечания

Примечание 1. Обратите внимание, что объявление пользовательской функции немного напоминает использование условного оператора `if` и циклов `for`, `while`.

Примечание 2. Объявление функции должно **предшествовать** ее вызову. Следующий программный код:

```
# вызов функции
print_message()

# объявление функции
def print_message():
    print('Я - Артур,')
    print('король британцев.')
```

приведет к ошибке:

```
NameError: name 'print_message' is not defined
```

Примечание 3. При объявлении функции следует убедиться, что каждая строка тела функции начинается с одинакового количества пробелов, иначе произойдет ошибка. Например, приведенное ниже определение функции приведет к ошибке:

```
def print_greeting():
    print('Доброе утро!')

print('Сегодня мы будем изучать функции.')
print('Это очень важная тема!')
```

Примечание 4. Иногда, при объявлении функции требуется сделать своего рода заглушку, чтобы функция ничего не выполняла. Тогда мы используем ключевое слово `pass`:

```
def do_nothing():
    pass
```

Мы объявили функцию с именем `do_nothing()`. Тело такой функции содержит единственную строку кода, которая ничего не делает.

Примечание 5. Функции часто называют **подпрограммами**.

Функции с параметрами

В предыдущем уроке мы определили функцию `draw_box()`, которая выводит звездный прямоугольник с размерами 5×7 :

```
def draw_box():  
    for i in range(5):  
        print('*' * 7)
```

Было бы намного удобнее, если бы функция `draw_box()` выводила прямоугольник с произвольными размерами. И действительно, функции могут принимать входные параметры, что делает их более гибкими.

Функции с параметрами объявляются так же как функции без параметров, только с указанием в скобках:

```
def название_функции(параметры):  
    блок кода
```

```
def имя функции ( параметры ) :  
    блок кода
```

Давайте перепишем предыдущую версию функции `draw_box()` так, чтобы она принимала параметры, задающие высоту и ширину прямоугольника:

```
def draw_box(height, width):    # функция принимает два параметра  
    for i in range(height):  
        print('*' * width)
```

Теперь наша функция `draw_box()` принимает два целочисленных параметра `height` – высота прямоугольника и `width` – ширина прямоугольника, и для ее вызова нам нужно обязательно их указать.

Чтобы вывести звездный прямоугольник размерами 5 на 7 мы пишем код:

```
draw_box(5, 7)
```

Результатом такого вызова функции `draw_box(5, 7)` будет:

```
*****  
*****  
*****  
*****  
*****
```

Чтобы вывести прямоугольник размерами 10 на 15, мы пишем код:

```
draw_box(10, 15)
```

Результатом такого вызова функции `draw_box(10, 15)` будет:

```
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
*****
```

Теперь с помощью нашей новой версии функции `draw_box()` можем в одной программе выводить прямоугольники разных размеров.

Приведённый ниже код:

```
draw_box(3, 3)
print()
draw_box(5, 5)
print()
draw_box(4, 10)
```

ВЫВОДИТ:

```
***
***
***

*****
*****
*****
*****
*****

*****
*****
*****
*****
```

На место параметров мы можем подставлять не только целочисленные константы, но и значения переменных.

Приведённый ниже код:

```
n = 3
m = 9
draw_box(n, m)
```

ВЫВОДИТ:

```
*****
*****
*****
```

Еще примеры

Напишем функцию `print_hello(n)`, которая принимает одно натуральное число n и печатает слово `Hello` ровно n раз.

```
def print_hello(n):  
    print('Hello' * n)
```

Приведённый ниже код:

```
print_hello(3)  
print_hello(5)  
times = 2  
print_hello(times)
```

ВЫВОДИТ:

```
HelloHelloHello  
HelloHelloHelloHelloHello  
HelloHello
```

Функцию `print_hello()` можно сделать более гибкой, если передавать в нее еще один параметр – текст для вывода:

```
def print_text(txt, n):  
    print(txt * n)
```

Приведённый ниже код:

```
print_text('Hello', 5)  
print_text('A', 10)
```

ВЫВОДИТ:

```
HelloHelloHelloHelloHello  
AAAAAAAAAAAA
```

Параметры VS аргументы

Аргумент – это любая порция данных, которая передается в функцию, когда функция вызывается. **Параметр** – это переменная, которая получает аргумент, переданный в функцию.

Для функции `draw_box(height, width)`:

```
def draw_box(height, width):  
    for i in range(height):  
        print('*' * width)
```


параметрами являются переменные `height` и `width` .

В момент вызова функции `draw_box(height, width)` :

```
height = 10
draw_box(height, 9)
```

аргументами являются `height` и `9` .



Параметры функций часто называют **параметрическими переменными**.

Внесение изменений в параметры

Когда аргумент передается в функцию, параметрическая переменная функции будет ссылаться на значение этого аргумента. Однако любые изменения, которые вносятся в параметрическую переменную, не будут влиять на аргумент.

Приведённый ниже код:

```
def draw_box(height, width):
    height = 2
    width = 10
    for i in range(height):
        print('*' * width)

n = 5
m = 7
draw_box(n, m)
print(n, m)
```

ВЫВОДИТ:

```
*****
*****
5 7
```

В теле функции вносятся изменения в значения параметрических переменных `height` и `width` , однако это никак не повлияло на значение переменных `n` и `m` из основной программы, которые передавались в качестве аргументов в функцию `draw_box()` .

Локальные переменные

Локальными называются переменные, объявленные внутри функции и доступные только ей самой. Программный код за пределами функции к ним доступа не имеет.

Рассмотрим функцию `print_texas()`, которая выводит информацию о количестве птиц, обитающих в Техасе.

```
def print_texas():  
    birds = 5000  
    print('В Техасе обитает', birds, 'птиц.')
```

В теле функции мы создаем переменную `birds`, которой присваивается значение, равное `5000`. Такая переменная является локальной для функции `print_texas()`. Всякий раз, когда переменной внутри функции присваивается значение, в результате **создаётся локальная переменная**. Она принадлежит функции, в которой создаётся, и к ней получает доступ только программный код этой функции.



Термин “**локальная**” указывает на то, что переменная может использоваться лишь в этом месте — внутри функции, в которой создается.

Если программный код одной функции попытается обратиться к локальной переменной, принадлежащей другой функции, произойдёт ошибка.

Рассмотрим следующий программный код:

```
def print_texas():  
    birds = 5000  
    print('В Техасе обитает', birds, 'птиц.')
```

```
def print_california():  
    print('В Калифорнии обитает', birds, 'птиц.')
```

Функция `print_california()` обращается к локальной переменной `birds` функции `print_texas()`. Вызов функции `print_california()`, приводит к ошибке:

```
NameError: name 'birds' is not defined
```

Локальные переменные скрыты от других функций, поэтому другие функции могут иметь собственные локальные переменные с тем же именем. Например,

```
def print_texas():  
    birds = 5000  
    print('В Техасе обитает', birds, 'птиц.')
```

```
def print_california():  
    birds = 9000  
    print('В Калифорнии обитает', birds, 'птиц.')
```

В каждой из этих двух функций есть локальная переменная с именем `birds`. Но они никогда не видны одновременно, так как находятся в разных функциях.

Когда выполняется функция `print_texas()`, видима переменная `birds`, значение которой равно `5000`. Когда выполняется функция `print_california()`, видима переменная `birds`, значение которой равно `9000`.



Разные функции могут иметь локальные переменные с одинаковыми именами, потому что они не видят локальных переменных друг друга.

Область видимости переменной

Область видимости переменной – часть программы, в которой можно к ней обращаться (та функция, где она создана). Переменная видима программному коду только в области её видимости. Никакая инструкция за пределами функции не может обращаться к такой переменной.

К локальной переменной не может обращаться программный код, который появляется внутри функции **до того**, как переменная была создана.

Например, если в функции `print_texas()` поменять местами строки кода:

```
def print_texas():  
    print('В Техасе обитает', birds, 'птиц.')  
    birds = 5000
```

то при вызове этой функции получим ошибку:

```
UnboundLocalError: local variable 'birds' referenced before assignment
```

Ошибка возникла в результате преждевременного обращения к еще не объявленной локальной переменной `birds`.

Область видимости параметрической переменной

Область видимости параметрической переменной – функция, в которой этот параметр используется. К параметрической переменной имеет доступ весь программный код этой функции.

Рассмотрим уже известную нам функцию:

```
def draw_box(height, width):  
    for i in range(height):  
        print('*' * width)
```

Параметрические переменные тут `height`, `width`. Внутри функции объявляется одна локальная переменная `i`.

Функция с возвратом значения

Функция с возвратом значения похожа на функцию без возврата значения тем, что:

- это набор инструкций, выполняющий определенную задачу;
- когда нужно выполнить функцию, ее вызывают.

Однако, когда функция с возвратом значения завершается, она возвращает значение в ту часть программы, которая ее вызвала. Возвращаемое из функции значение используется как любое другое: оно может быть присвоено переменной, выведено на экран, использовано в математическом выражении (если это число) и т. д.



Функция с возвратом значения возвращает значение обратно в ту часть программы, которая ее вызвала.

Мы уже сталкивались со многими функциями с возвратом значений:

- функция `int()` – преобразует строку к целому числу и возвращает его;
- функция `float()` – преобразует строку к вещественному числу и возвращает его;
- функция `range()` – возвращает последовательность целых чисел `0, 1, 2, ...`;
- функция `abs()` – возвращает абсолютное значение числа (модуль числа);
- функция `len()` – возвращает длину строки или списка.

Функцию с возвратом значения пишут точно так же, как и без, но она должна иметь инструкцию `return`.

Вот общий формат определения функции с возвратом значения в Python:

```
def название_функции():  
    блок кода  
    return выражение
```

В функции должна быть инструкция `return`, принимающая форму:

```
return выражение
```

Значение выражения, которое следует за ключевым словом `return`, будет отправлено в ту часть программы, которая вызвала функцию. Это может быть переменная либо выражение, к примеру, математическое.



Функция с возвратом значения имеет инструкцию `return`, возвращающую значение в ту часть программы, которая ее вызвала.

При изучении вещественных чисел мы решали задачу о переводе градусов по шкале Фаренгейта в градусы по шкале Цельсия по формуле $C = \frac{5}{9} (F - 32)$.

Напишем функцию, которая осуществляет перевод:

```
def convert_to_celsius(temp):  
    result = (5 / 9) * (temp - 32)  
    return result
```

Задача этой функции — принять одно число `temp` в качестве аргумента — количество градусов по шкале Фаренгейта, и вернуть другое — количество градусов по шкале Цельсия.

Рассмотрим ее работу. Первая инструкция в блоке функции присваивает значение `(5 / 9) * (temp - 32)` переменной `result`. Затем выполняется инструкция `return`, которая приводит к завершению исполнения функции и отправляет значение из переменной `result`, назад в ту часть программы, которая вызвала эту функцию.

```
# функция перевода градусов Фаренгейта в градусы Цельсия  
def convert_to_celsius(temp):  
    result = (5 / 9) * (temp - 32)  
    return result  
  
# основная программа  
temp = float(input('Введите количество градусов по Фаренгейту: '))  
celsius = convert_to_celsius(temp)  
print(celsius) # градусы Цельсия
```

Основная программа получает от пользователя одно число — значение в градусах Фаренгейта, и вызывает функцию, передавая значение переменной `temp` в качестве аргумента. Значение, которое возвращается из функции `convert_to_celsius`, присваивается переменной `celsius`.

функция ()

функция (параметр)

переменная = функция ()

переменная = функция (параметр)

Использование инструкции return по максимуму

Взглянем еще раз на функцию `convert_to_celsius()`:

```
def convert_to_celsius(temp):  
    result = (5 / 9) * (temp - 32)  
    return result
```

Обратите внимание, что внутри этой функции происходят две вещи: во-первых, переменной `result` присваивается значение выражения `(5 / 9) * (temp - 32)`, и во-вторых, значение переменной `result` возвращается из функции. Эта функция хорошо справляется с поставленной перед ней задачей, но ее можно упростить. Поскольку инструкция `return` возвращает значение выражения, переменную `result` устраним и перепишем функцию так:

```
def convert_to_celsius(temp):  
    return (5 / 9) * (temp - 32)
```

Эта версия функции не сохраняет значение `(5 / 9) * (temp - 32)` в отдельной переменной, а сразу возвращает значение выражения с помощью инструкции `return`. Делает то же, что и предыдущая версия, но за один шаг.

Использование нескольких return

В одной функции может быть сколько угодно инструкций `return`. Рассмотрим функцию `convert_grade()`, которая переводит стобалльную оценку в пятибалльную:

```
def convert_grade(grade):  
    if grade >= 90:  
        return 5  
    elif grade >= 80:  
        return 4  
    elif grade >= 70:  
        return 3  
    elif grade >= 60:  
        return 2  
    else:  
        return 1  
  
# основная программа  
grade = int(input('Введите вашу отметку по 100-балльной системе: '))  
print(convert_grade(grade))
```

В функции `convert_grade()` используется 5 инструкций `return`. Каждая из них возвращает соответствующее значение и завершает работу функции.

Функцию `convert_grade()` можно переписать с помощью одной инструкции `return`:

```
def convert_grade(grade):  
    if grade >= 90:  
        result = 5  
    elif grade >= 80:  
        result = 4  
    elif grade >= 70:  
        result = 3  
    elif grade >= 60:  
        result = 2  
    else:  
        result = 1  
  
    return result
```

Функции с возвратом нескольких значений

В Python функции не ограничены возвратом всего одного значения. После инструкции `return` можно определить много выражений, разделенных запятыми:

```
return выражение 1, выражение 2, выражение 3 ...
```

Следующий программный код определяет функцию `get_powers(num)`, которая принимает в качестве аргумента число `num` и возвращает его квадрат, куб и четвертую степень.

```
def get_powers(num):  
    return num**2, num**3, num**4
```

Приведённый ниже код:

```
a, b, c = get_powers(2)  
print(a)  
print(b)  
print(c)
```

ВЫВОДИТ:

```
4  
8  
16
```



Возвращающие несколько значений функции – исключительная особенность языка Python. В большинстве языков программирования для возврата нескольких значений используется результирующий тип данных – список, который может содержать несколько значений.

Рассмотрим ещё один пример. Пусть требуется написать функцию, которая находит точку пересечения двух **непараллельных** прямых $ax + by = e$ и $cx + dy = f$. Другими словами требуется решить систему уравнений:

$$\begin{cases} ax + by = e \\ cx + dy = f \end{cases}$$

Несложно прийти к решению этой системы:

$$\begin{cases} ax + by = e \\ cx + dy = f \end{cases} \iff \begin{cases} x = \frac{d \cdot e - b \cdot f}{a \cdot d - b \cdot c} \\ y = \frac{a \cdot f - c \cdot e}{a \cdot d - b \cdot c} \end{cases}$$

Программный код, решающий задачу, имеет вид:

```
def solve(a, b, c, d, e, f):  
    x = (d * e - b * f) / (a * d - b * c)  
    y = (a * f - c * e) / (a * d - b * c)  
    return x, y
```

Приведённый ниже код:

```
xsol, ysol = solve(2, 3, 4, 1, 2, 5)  
print('Решением системы являются числа', 'x =', xsol, 'y =', ysol)
```

ВЫВОДИТ:

```
Решением системы являются числа x = 1.3 y = -0.2
```


